

TECHNICAL PROJECT REPORT

- Project Title:- Project FORESIGHT – AI-Powered Demand & Inventory Intelligence Platform.

- Author / Developer:- Pankaj kumar Yadav

- Deployment URL:- <https://project---foresight-dz7b4onwys5h8a2s6jedmq.streamlit.app/>

- Github Repository Source:- <https://github.com/pankaj-1212/Project---Foresight.git>

1. Project Overview & Business Value:-

1.1 Executive Summary :-

- Project FORESIGHT is an enterprise-grade predictive analytics platform designed to optimize supply chain pipelines and inventory allocation. By applying machine learning models to transactional data, the application transforms reactive historical records into proactive, forward-looking insights. The primary objective is to maximize capital efficiency by minimizing dead stock (over-stocking) and preventing revenue loss from out-of-stock scenarios (stockouts).

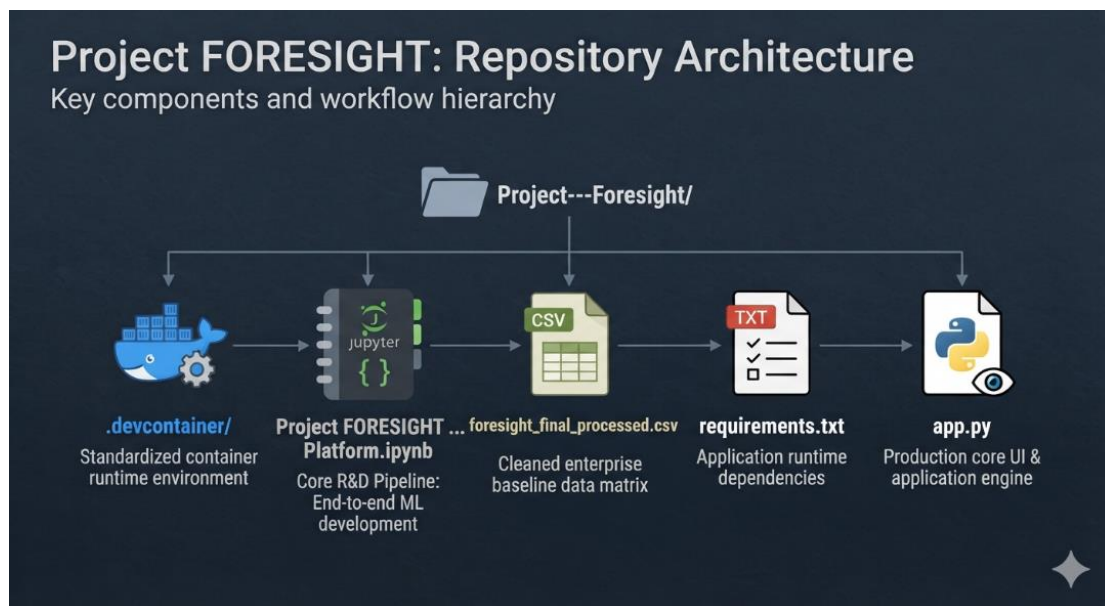
1.2 Core Business Metrics Addressed:-

- The application acts as a single source of truth for supply chain visibility, actively tracking three primary data nodes on its live dashboard:

- Total Revenue Tracked: \$1,989,984.85
- Total Inventory Volume Processed: 24,437 Units
- Logistical Baseline (Average Supplier Delivery Speed): 3.7 Days.

2. Technical Stack & Architecture:-

- The project is architected completely in Python, optimized for light infrastructure overhead, rapid data processing, and simple web deployment.

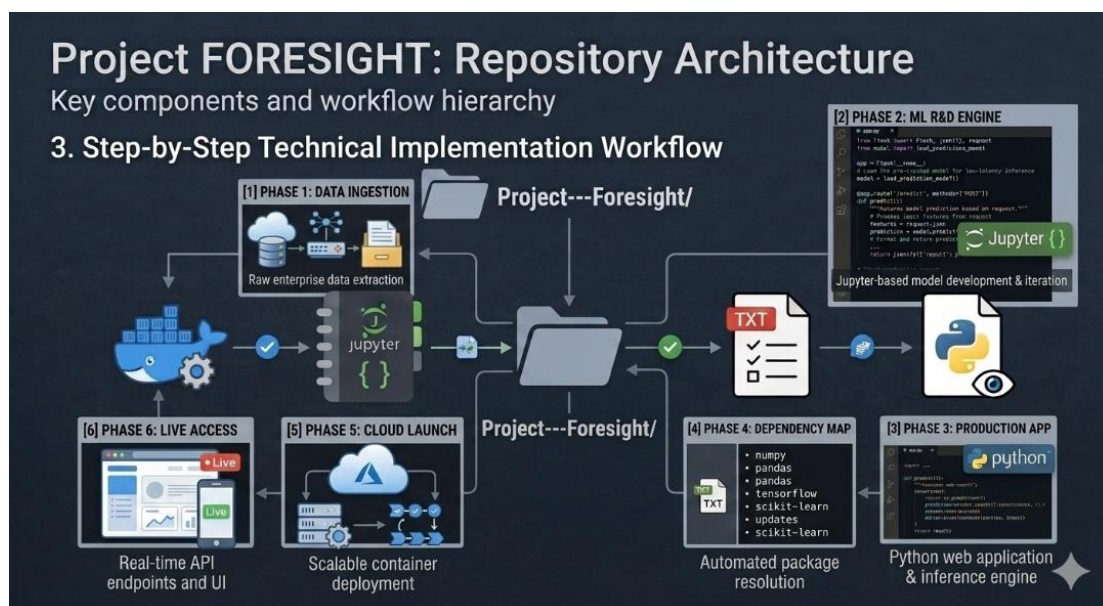


2.1 Technology Profile:-

- Engine Core: Python 3.
- User Interface Layer: Streamlit Community Cloud Framework.
- Mathematical Processing: Pandas & NumPy Data Structuring Libraries.
- Model Engineering: Scikit-Learn Framework & Jupyter Lab Sub-environments.

3. Step-by-Step Technical Implementation Workflow:-

- This section outlines the exact phase-by-phase execution of the project, detailing how the analytical engine was built inside the Jupyter Notebook (.ipynb) and translated into a live application (app.py).



Step 1: Load Dataset:-

- Execution: Automated ingestion of the primary raw operational files using `pandas.read_csv()`.
- Objective: Read the structural transaction data into memory as a unified DataFrame to establish the baseline training matrix.

Step 2: Data Preprocessing (Filtering & Feature Engineering):-

- Filtering: Structured queries were applied to remove data anomalies, handle missing records, and isolate the targeted operational bounds.
- Feature Engineering: Chronological timestamps were broken down into distinct mathematical components (Day, Month, Quarter) to extract seasonal purchasing patterns and cyclical buying waves.

Step 3: Inventory Logic & ML Preparation:-

- Inventory Formulation: Built specialized retail logic calculations directly into the data matrix, accounting for stock velocity, lead times, and the 3.7-day supplier delivery baseline.
- ML Prep: Split the data frames into feature matrices ((X)) and target vectors ((y)), followed by cross-validation partitioning (Train/Test splits) to protect against model overfitting.

Step 4: Machine Learning Model Training:-

- Execution: Initialized and trained supervised machine learning regression algorithms via scikit-learn.
- Objective: Train the model to map historical seasonal variations to precise consumer quantity demands.

Step 5: Data Visualization (Exploratory Data Analysis - EDA Graph):-

- Execution: Generated multi-dimensional graphical layouts using matplotlib and seaborn.
- Objective: Visually audit demand distribution curves, identify structural anomalies, and analyze correlation heatmaps before deploying the data models.

Step 6: Hyperparameter Tuning (Model Optimization):-

- Execution: Applied programmatic search strategies (e.g., GridSearchCV or RandomizedSearchCV).
- Objective: Systematically iterate through algorithm configurations to discover optimal model parameters, minimizing root-mean-squared error (RMSE) metrics.

Step 7: Feature Importance Analysis:-

- Execution: Extracted and plotted internal model coefficient weights (feature_importances_).
- Objective: Identify the highest-impact drivers behind product

demand fluctuations, giving executives full transparency into why the AI outputs specific recommendations.

Step 8: Production UI Scripting (app.py):-

- Execution: Ported the optimized modeling variables and the final dataset matrix (foresight_final_processed.csv) into the production web script.
- Objective: Construct a lightweight dashboard featuring a global sidebar filter and a responsive Actual Historical Demand vs. AI-Predicted Demand Curve canvas.

Step 9: Cloud Deployment:-

- Execution: Configured dependency files (requirements.txt), linked the GitHub repository to Streamlit Community Cloud, and launched the runtime server.
- Objective: Provide stakeholders with instant, 24/7 web-based access to live platform metrics.

4. Strategic Development Roadmap

- To scale Project FORESIGHT from its current prototype state into an automated enterprise resource planning (ERP) component, the following upgrades are recommended:

- Phase 1 (Data Layer):-**

Transition the local data pipeline away from flat file sources toward a live cloud data warehouse infrastructure (such as PostgreSQL, Snowflake, or AWS Redshift).

- Phase 2 (Modeling Layer):-**

Swap out baseline regression models for advanced deep learning time-series architectures (such as NeuralProphet or DeepAR) to better isolate macro-economic fluctuations.

- Phase 3 (Automation Layer):-**

Build automated webhooks that alert purchasing teams or automatically flag vendors when predicted demand patterns dip below safe storage limits.

Thank you for your time and consideration